WEEK-4

Aim: Implement Dijsktra's algorithm to compute the shortest path through a network

```
#include<stdio.h>
#define MAX 10
#define INF 9999
int main()
{
   int cost[MAX][MAX], dist[MAX], visited[MAX], parent[MAX];
   int n, source, destination;
   int i, j, min, next;
   int path[MAX], count = 0;

   printf("Enter number of vertices: ");
   scanf("%d",&n);

   printf("Enter the cost matrix:\n");

   for(i=0;i<n;i++)
   {
      for(j=0;j<n;j++)
      {
         scanf("%d",&cost[i][j]);

         if(cost[i][j]==0 && i!=j)
            cost[i][j]=INF;
      }
   }

   printf("Enter source vertex: ");
   scanf("%d",&source);

   printf("Enter destination vertex: ");
```

```
scanf("%d",&destination);

// Initialization
for(i=0;i<n;i++)
{
   dist[i]=cost[source][i];
   visited[i]=0;

   if(cost[source][i]!=INF)
      parent[i]=source;
   else
      parent[i]=-1;
}

dist[source]=0;
visited[source]=1;
parent[source]=-1;

// Dijkstra Algorithm
for(i=1;i<n;i++)
{
   min=INF;
   next=-1;

   // Find nearest unvisited vertex
   for(j=0;j<n;j++)
   {
      if(visited[j]==0 && dist[j]<min)
      {
         min=dist[j];
         next=j;
      }
```

```
    }

    if(next==-1)
        break;

    visited[next]=1;

    // Update distances
    for(j=0;j<n;j++)
    {
        if(visited[j]==0 &&
           cost[next][j]!=INF &&
           dist[next]+cost[next][j]<dist[j])
        {
            dist[j]=dist[next]+cost[next][j];
            parent[j]=next;
        }
    }
}

if(dist[destination]==INF)
{
    printf("\nNo path exists.");
}
else
{
    printf("\nMinimum Cost = %d\n",dist[destination]);

    // Store the path in reverse order
    i=destination;

    while(i!=-1)
```

```
        {
            path[count]=i;
            count++;
            i=parent[i];
        }

        printf("Shortest Path: ");

        // Print in correct order
        for(i=count-1;i>=0;i--)
        {
            printf("%d",path[i]);

            if(i!=0)
                printf(" -> ");
        }
    }

    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the cost matrix:
0 2 7 8
2 0 1 5
7 1 0 2
8 5 2 0
Enter source vertex: 0
Enter destination vertex: 2

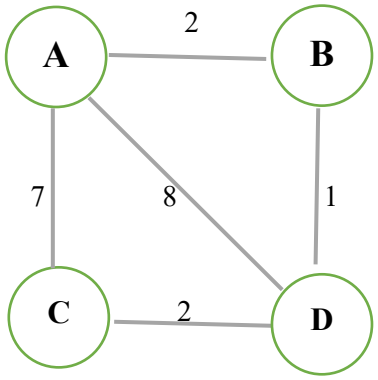
Minimum Cost = 3
Shortest Path: 0 -> 1 -> 2
```

Theory:

Dijkstra's Algorithm is used to find the shortest path from a source vertex to a destination vertex or to all other vertices in a weighted graph. It is widely used in Computer Networks for finding the optimum route between routers. In the graph, each vertex represents a router and each edge represents a communication link with an associated cost (weight).

The algorithm starts from the source vertex, selects the unvisited vertex with the minimum cost, updates the costs of its neighbouring vertices if a shorter path is found, and repeats the process until the destination is reached or all vertices are visited.

Ex:



Consider **A** as the source vertex and **D** as the destination vertex.

Direct Path: A → D = 8

Path 1: A → B → D = 2 + 1 = 3

Path 2: A → C → D = 7 + 2 = 9

Among all the possible paths, the minimum cost is 3. Therefore, the shortest path selected by Dijkstra's Algorithm is:

A → B → D

Minimum Cost = 3

Thus, Dijkstra's Algorithm always finds the path with the minimum total cost, making it useful for efficient routing in computer networks.